



인덱스 Dive Deep

젠슨

Product

id (pk)

category_id (fk)

price

name

product_status

created_at

```
SELECT *  
FROM product  
WHERE category_id = 11  
      AND price BETWEEN 10000 AND 50000  
      AND product_status = 'ACTIVE'  
ORDER BY created_at  
LIMIT 20;
```

활성화 + 10,000 ~ 50,000원 가격대 상품 중에서,
과거에 생성된 상품 순으로 상위 20개를 뽑아주세요.

인덱스

Product

id (pk)

category_id (fk)

price

name

product_status

created_at

```
SELECT *
FROM product
WHERE category_id = 11
      AND price BETWEEN 10000 AND 50000
      AND product_status = 'ACTIVE'
ORDER BY created_at
LIMIT 20;
```

[illegible]

```
-> Limit: 20 row(s) (cost=62818 rows=20)
    (actual time=682..682 rows=20 loops=1)
-> Sort: product.created_at, limit input to 20 row(s) per chunk
    (cost=62818 rows=413612) (actual time=682..682 rows=20 loops=1)
-> Filter: ((product.price between 10000 and 50000) and (product.product_status = 'ACTIVE'))
    (cost=62818 rows=413612) (actual time=0.5..671 rows=91871 loops=1)
-> Index lookup on product using idx_product_category_id (category_id=11)
    (cost=62818 rows=413612) (actual time=0.446..657 rows=200829 loops=1)
```

인덱스

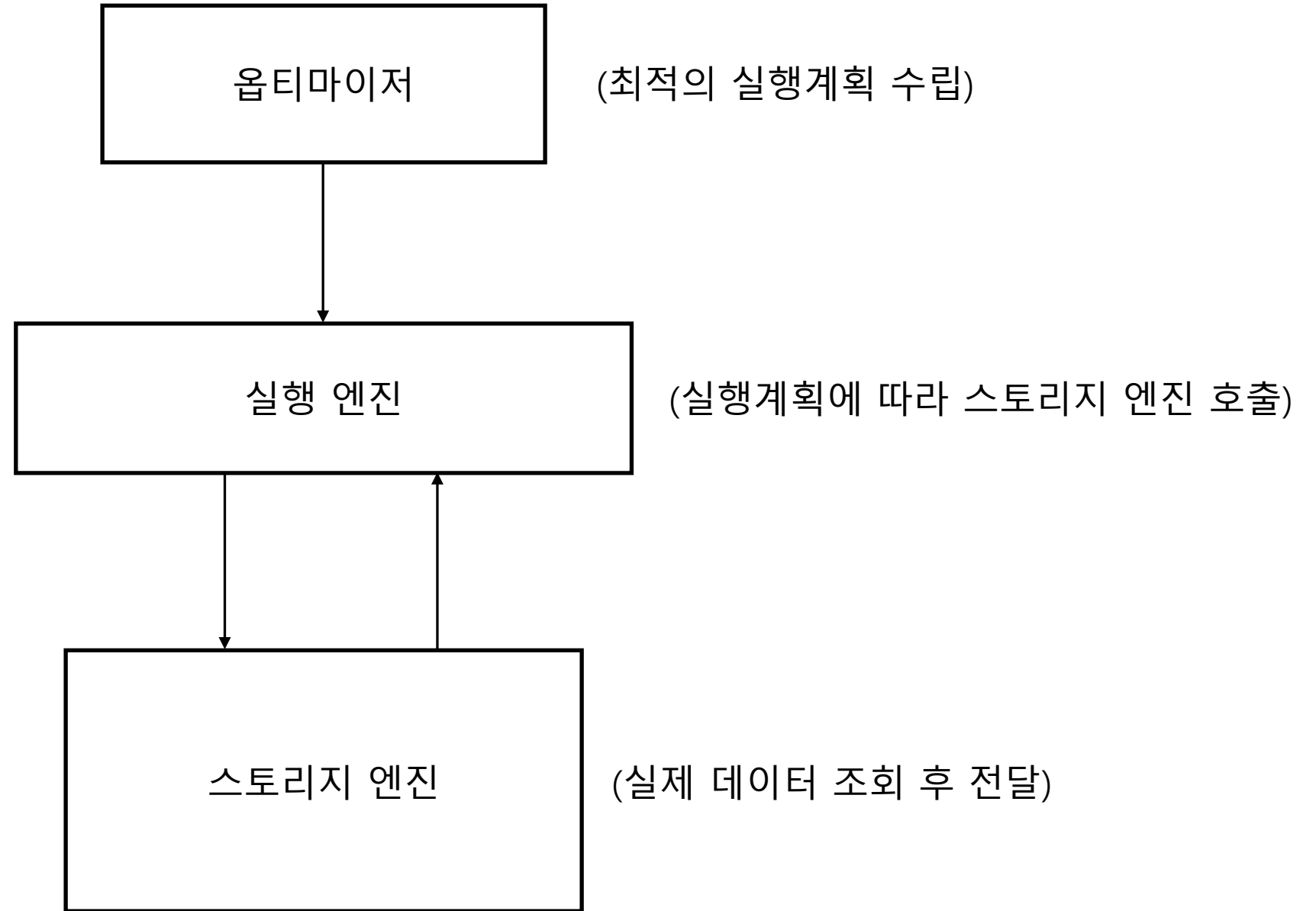
```
SELECT *  
FROM product  
WHERE category_id = 11  
      AND price BETWEEN 10000 AND 50000  
      AND product_status = 'ACTIVE'  
ORDER BY created_at  
LIMIT 20;
```

1. (category_id, product_status, created_at)
2. (category_id, product_status, price, created_at)
3. (category_id, product_status, created_at, price)

인덱스 튜닝을 할 때 가장 중요한 것

- 1) 감으로 대응하면 안 된다
- 2) 조기 종료가 가능한가?

MySQL의 탐색 구조



MySQL의 탐색 구조

```
SELECT *  
FROM product  
WHERE product_status = 'ACTIVE';
```

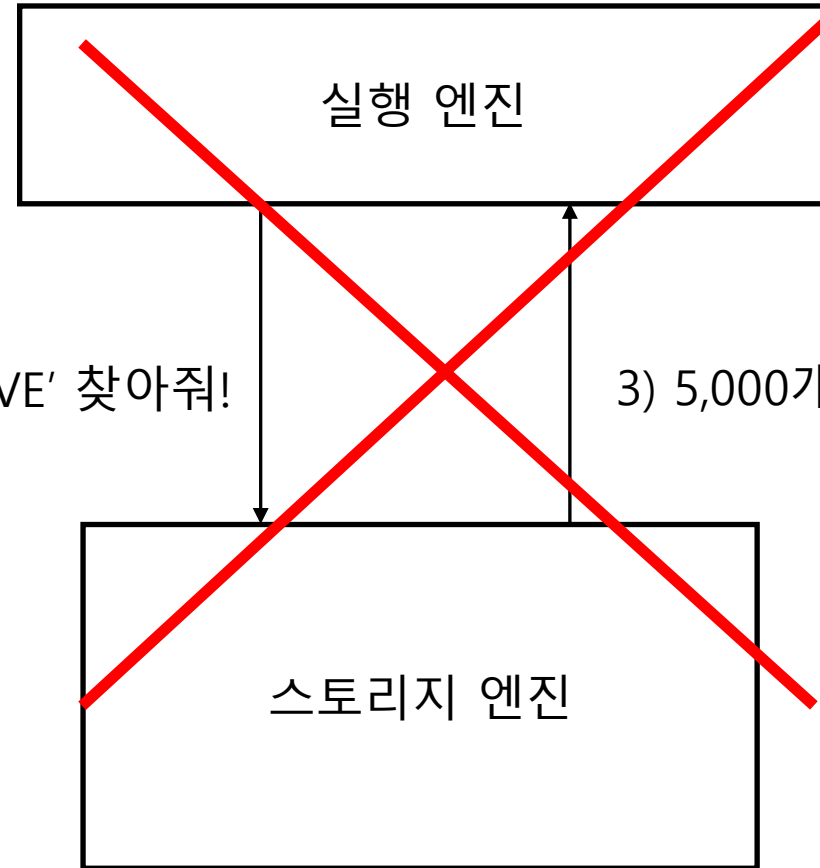
1) product_status = 'ACTIVE' 찾아줘!

실행 엔진

3) 5,000개 행 반환!

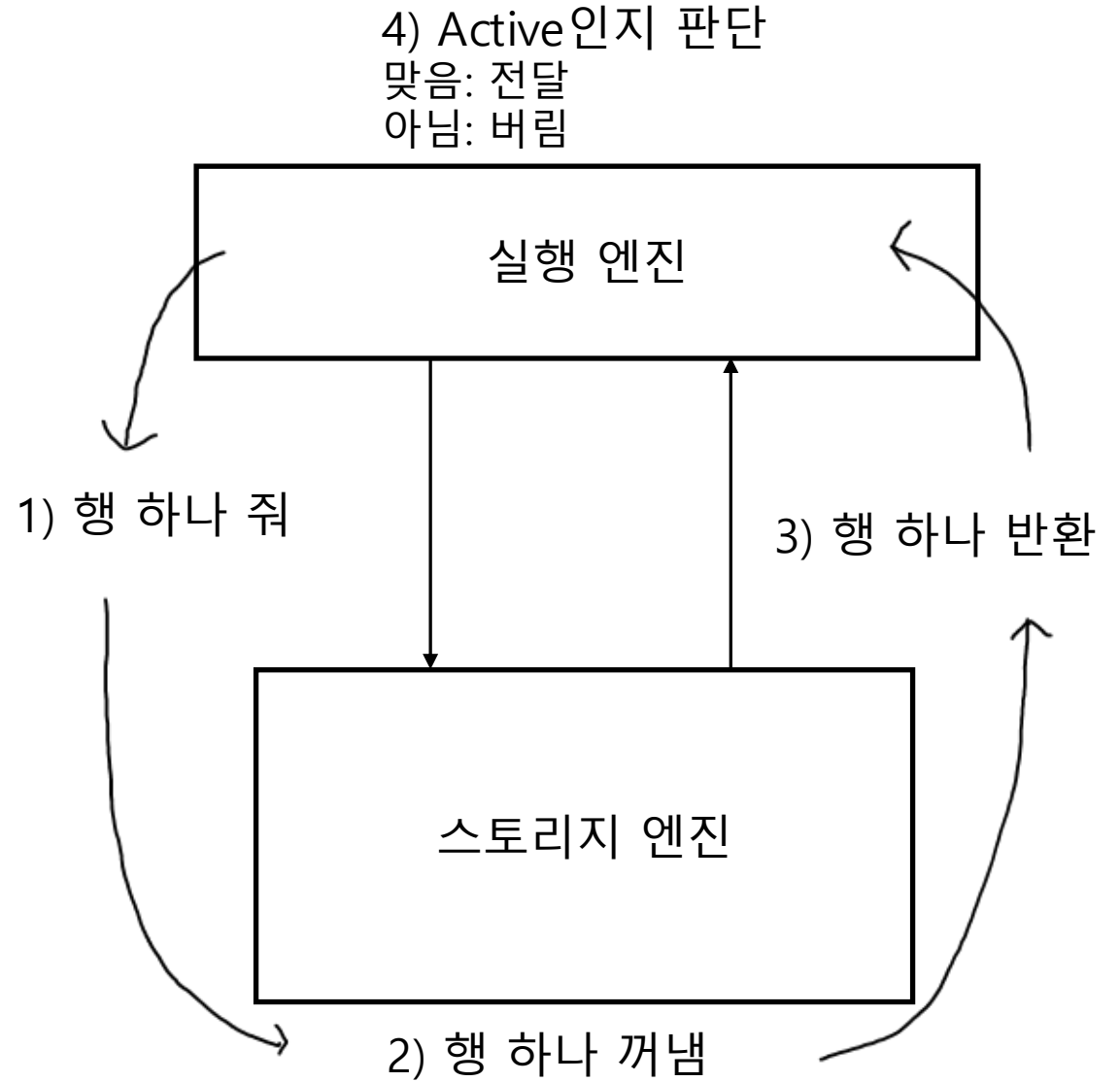
스토리지 엔진

2) Active인 행이
5,000개네?



MySQL의 탐색 구조

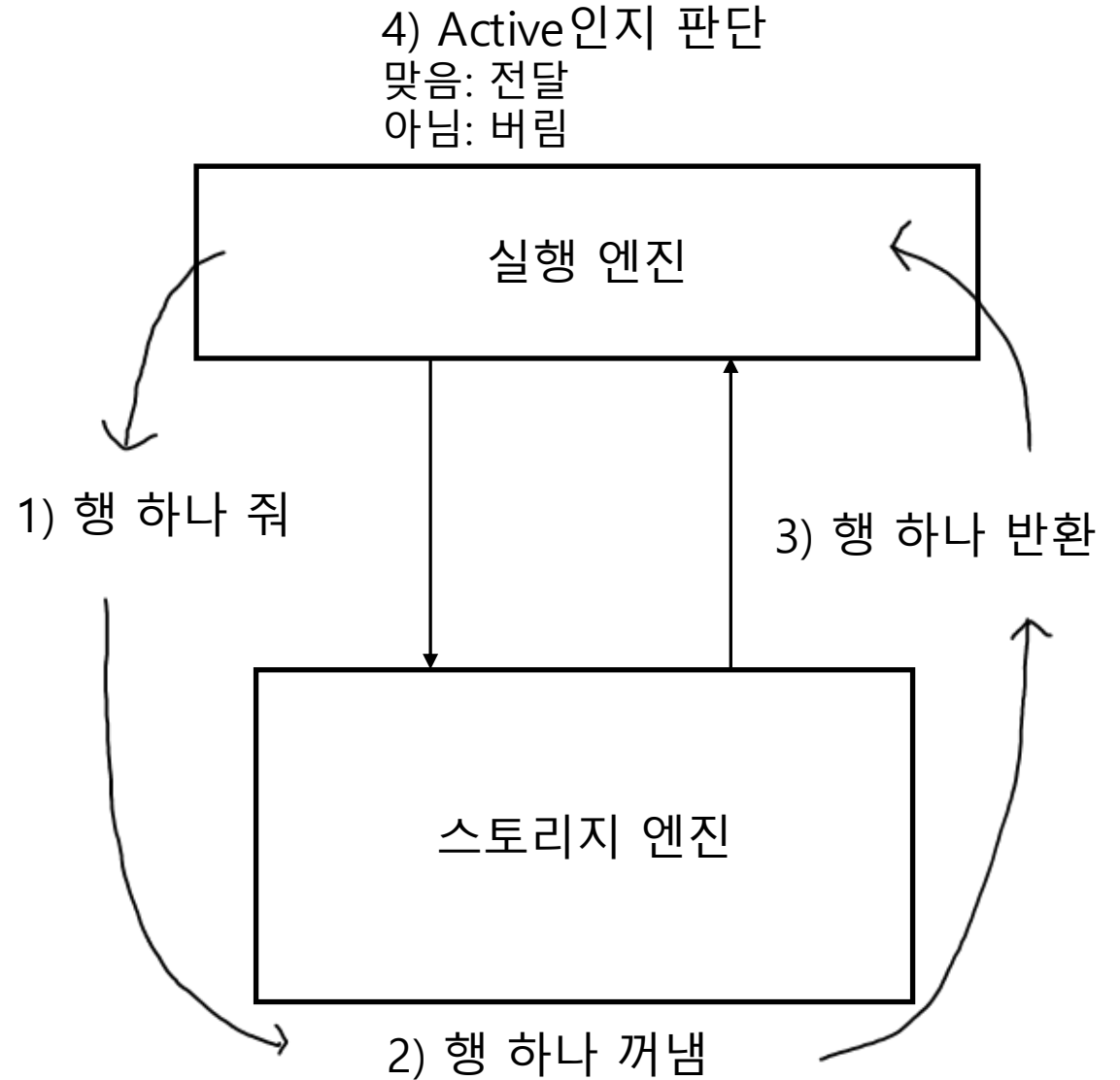
```
SELECT *  
FROM product  
WHERE product_status = 'ACTIVE';
```



MySQL의 탐색 구조

```
SELECT *  
FROM product  
WHERE product_status = 'ACTIVE'  
LIMIT 20;
```

20개를 찾으면 루프 종료!



MySQL의 탐색 구조

```
SELECT *  
FROM product  
WHERE product_status = 'ACTIVE'  
ORDER BY created_at  
LIMIT 20;
```

- 오래된 순 20개 정렬
- TOP-N

실행 엔진

데이터가 전부 필요해...!

MySQL의 탐색 구조

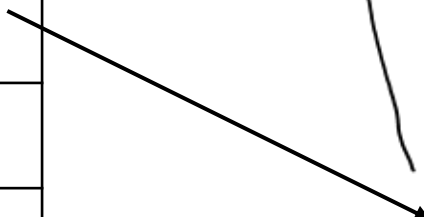
created_at에 인덱스가 걸려있다면?

세컨더리 인덱스

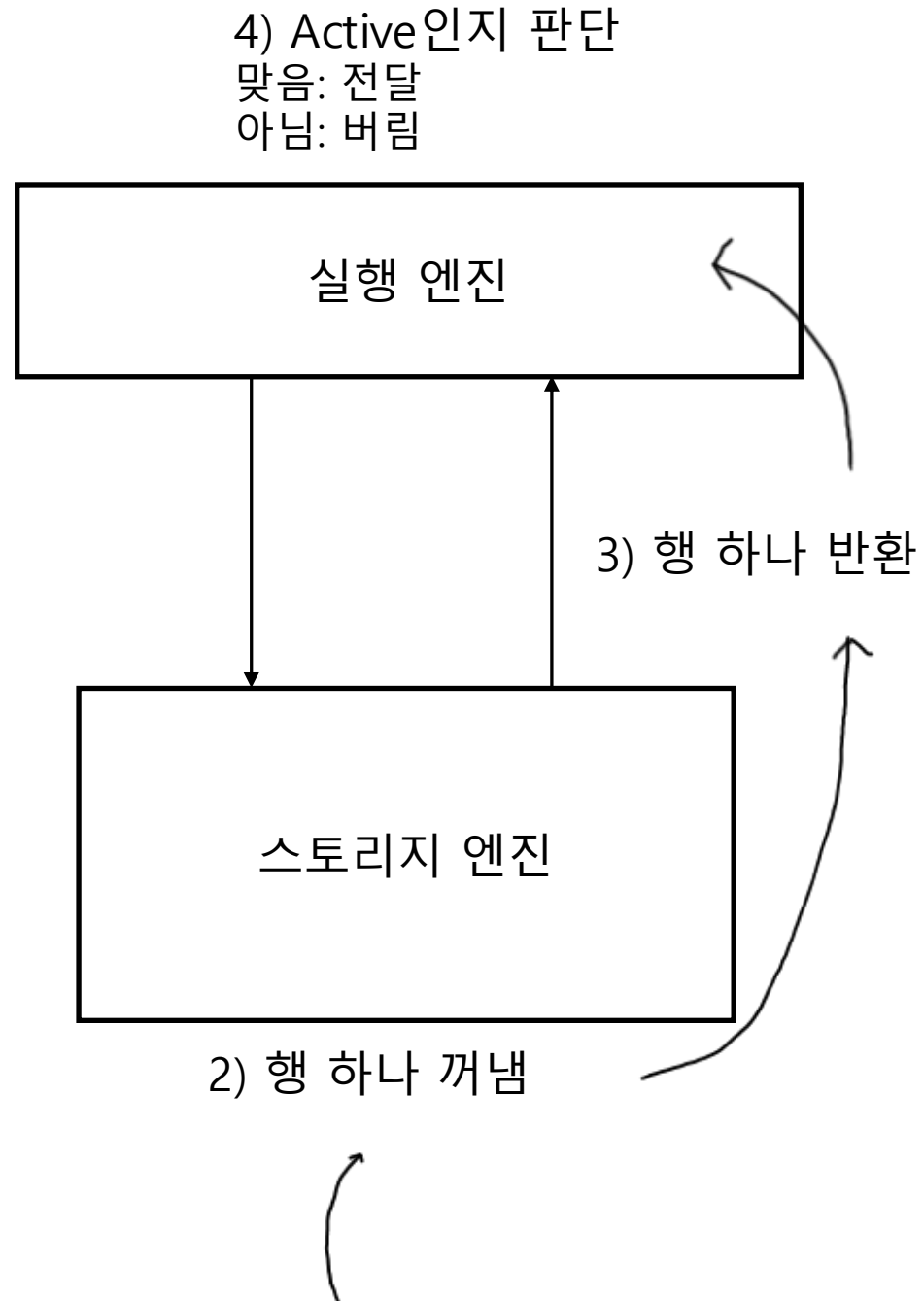
created_at	pk
2026-08-26	3
2026-08-27	1
2026-08-28	2
2026-08-29	4
2026-08-30	6
2026-08-31	5

클러스터링 인덱스

pk	product_status	created_at
1	ACTIVE	2026-08-27
2	ACTIVE	2026-08-28
3	UNACTIVE	2026-08-26
4	ACTIVE	2026-08-29
5	ACTIVE	2026-08-31
6	ACTIVE	2026-08-30



MySQL의 탐색 구조



정렬이 필요없기 때문에
조기 종료가 가능하다!

인덱스

```
SELECT *  
FROM product  
WHERE category_id = 11  
      AND price BETWEEN 10000 AND 50000  
      AND product_status = 'ACTIVE'  
ORDER BY created_at  
LIMIT 20;
```

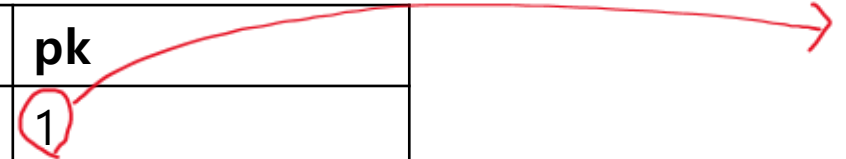
1. (category_id, product_status, created_at)
2. (category_id, product_status, price, created_at)
3. (category_id, product_status, created_at, price)

인덱스

1. (category_id, product_status, created_at)

세컨더리 인덱스

category_id	product_status	created_at	pk
11	ACTIVE	2026-08-27	1
11	ACTIVE	2026-08-28	2
11	ACTIVE	2026-08-29	4
11	ACTIVE	2026-08-30	6
11	ACTIVE	2026-08-31	5
11	UNACTIVE	2026-08-27	3
11	UNACTIVE	2026-08-29	10



인덱스

클러스터링 인덱스



pk	category_id	product_status	name	price	created_at
1	11	ACTIVE	신발1	24,000	2026-08-27
2	11	ACTIVE	신발3	12,000	2026-08-28
3	11	UNACTIVE	신발5	15,000	2026-08-27
4	11	ACTIVE	신발2	45,000	2026-08-29
5	11	ACTIVE	신발10	9,000	2026-08-31
6	11	ACTIVE	신발9	24,000	2026-08-30
7	11	ACTIVE	신발7	33,000	2026-08-25

MySQL의 탐색 구조

Filter: price가 10,000 ~ 50,000 범위 안에 속하는가?

```
SELECT *  
FROM product  
WHERE category_id = 11  
      AND price BETWEEN 10000 AND 50000  
      AND product_status = 'ACTIVE'  
ORDER BY created_at  
LIMIT 20;
```

pk	category_id	product_status	name	price	created_at
1	11	ACTIVE	신발1	24,000	2026-08-27

실행 엔진

조기 종료가 가능한 인덱스

인덱스

2. (category_id, product_status, price, created_at)

세컨더리 인덱스

category_id	product_status	price	created_at	pk
11	ACTIVE	9,000	2026-08-31	5
11	ACTIVE	12,000	2026-08-28	2
11	ACTIVE	24,000	2026-08-27	1
11	ACTIVE	24,000	2026-08-30	6
11	ACTIVE	45,000	2026-08-29	4
11	UNACTIVE	8,000	2026-08-29	10
11	UNACTIVE	15,000	2026-08-27	3

인덱스

2. (category_id, product_status, price, created_at)

```
SELECT *  
FROM product  
WHERE category_id = 11  
      AND price BETWEEN 10000 AND 50000  
      AND product_status = 'ACTIVE'  
ORDER BY created_at  
LIMIT 20;
```

price	created_at	pk
9,000	2026-08-31	5
12,000	2026-08-28	2
24,000	2026-08-27	1
24,000	2026-08-30	6
45,000	2026-08-29	4

Index Range Scan으로 price 10,000~ 50,000을 전부 탐색해야함
조기종료 불가

인덱스

3. (category_id, product_status, created_at, price)

3번을 분석하기 전에 알아야 할 사전 지식

ICP (Index Condition Pushdown)

인덱스

3. (category_id, product_status, created_at, price)

세컨더리 인덱스

category_id	product_status	created_at	price	pk
11	ACTIVE	2026-08-27	24,000	1
11	ACTIVE	2026-08-28	12,000	2
11	ACTIVE	2026-08-29	45,000	4
11	ACTIVE	2026-08-30	24,000	6
11	ACTIVE	2026-08-31	9,000	5
11	UNACTIVE	2026-08-27	15,000	3
11	UNACTIVE	2026-08-20	8,000	10

3. (category_id, product_status, created_at, price)

category_id	product_status	created_at	price	pk
11	ACTIVE	2026-08-27	24,000	1
11	ACTIVE	2026-08-28	12,000	2
11	ACTIVE	2026-08-29	45,000	4
11	ACTIVE	2026-08-30	24,000	6
11	ACTIVE	2026-08-31	9,000	5
11	UNACTIVE	2026-08-27	15,000	3
11	UNACTIVE	2026-08-20	8,000	10

인덱스

3. (category_id, product_status, created_at, price)

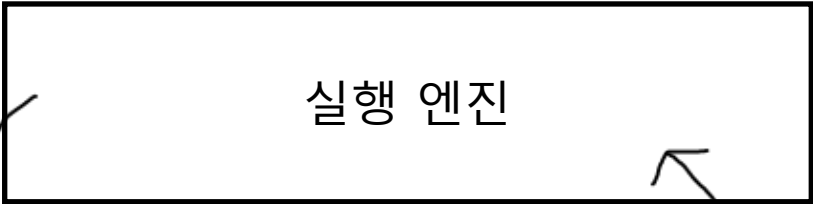


pk	category_id	product_status	name	price	created_at
1	11	ACTIVE	신발1	24,000	2026-08-27
2	11	ACTIVE	신발3	12,000	2026-08-28
3	11	UNACTIVE	신발5	15,000	2026-08-27
4	11	ACTIVE	신발2	45,000	2026-08-29
5	11	ACTIVE	신발10	9,000	2026-08-31
6	11	ACTIVE	신발9	24,000	2026-08-30
7	11	ACTIVE	신발7	33,000	2026-08-25

MySQL의 탐색 구조

4) price 가격 판단
맞음: 전달
아님: 버림

조건 판단은 실행 엔진이 담당

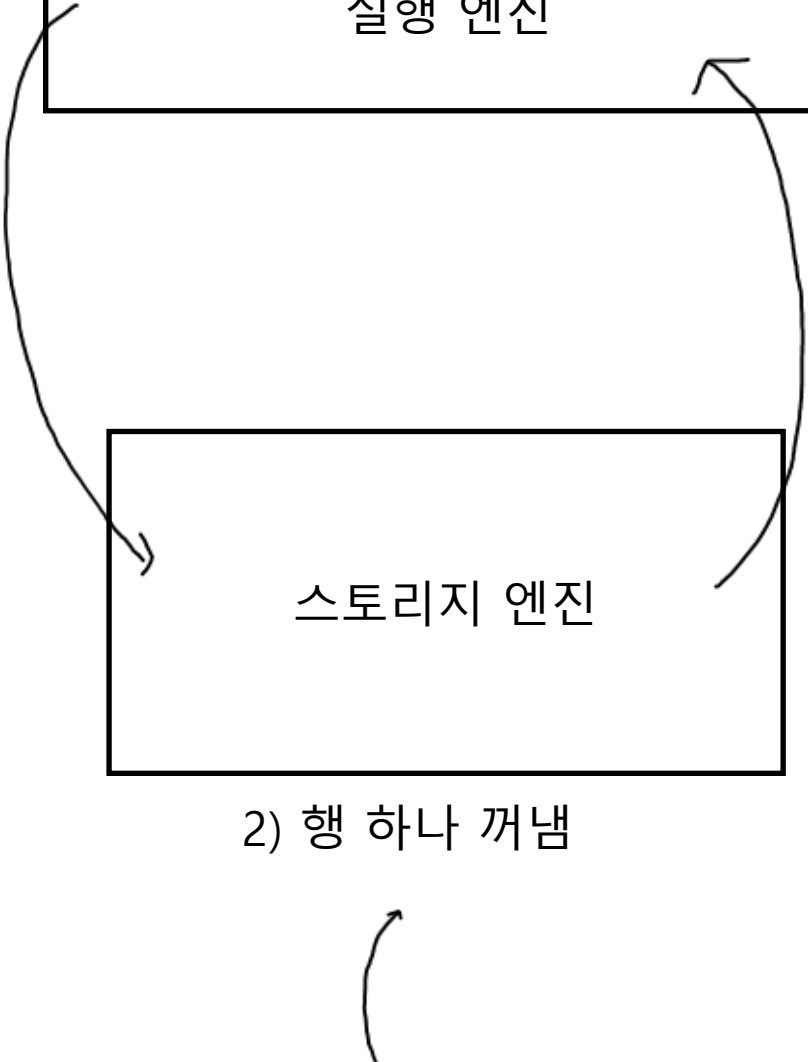


5) 다음 행 요청

3) 행 하나 반환

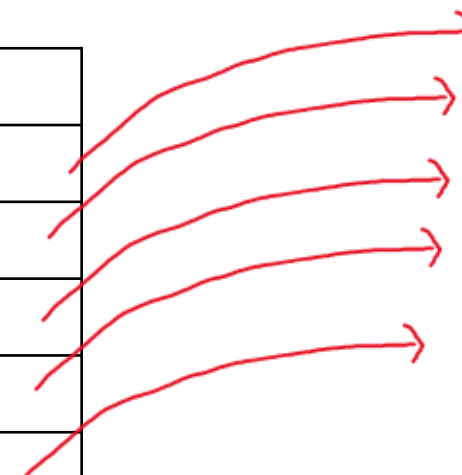


2) 행 하나 꺼냄



2. (category_id, product_status, created_at, price)

category_id	product_status	created_at	price	pk
11	ACTIVE	2026-08-27	24,000	1
11	ACTIVE	2026-08-28	12,000	2
11	ACTIVE	2026-08-29	45,000	4
11	ACTIVE	2026-08-30	24,000	6
11	ACTIVE	2026-08-31	9,000	5
11	UNACTIVE	2026-08-27	15,000	3
11	UNACTIVE	2026-08-20	8,000	10



5번의 Random I/O 발생

2. (category_id, product_status, created_at, price)

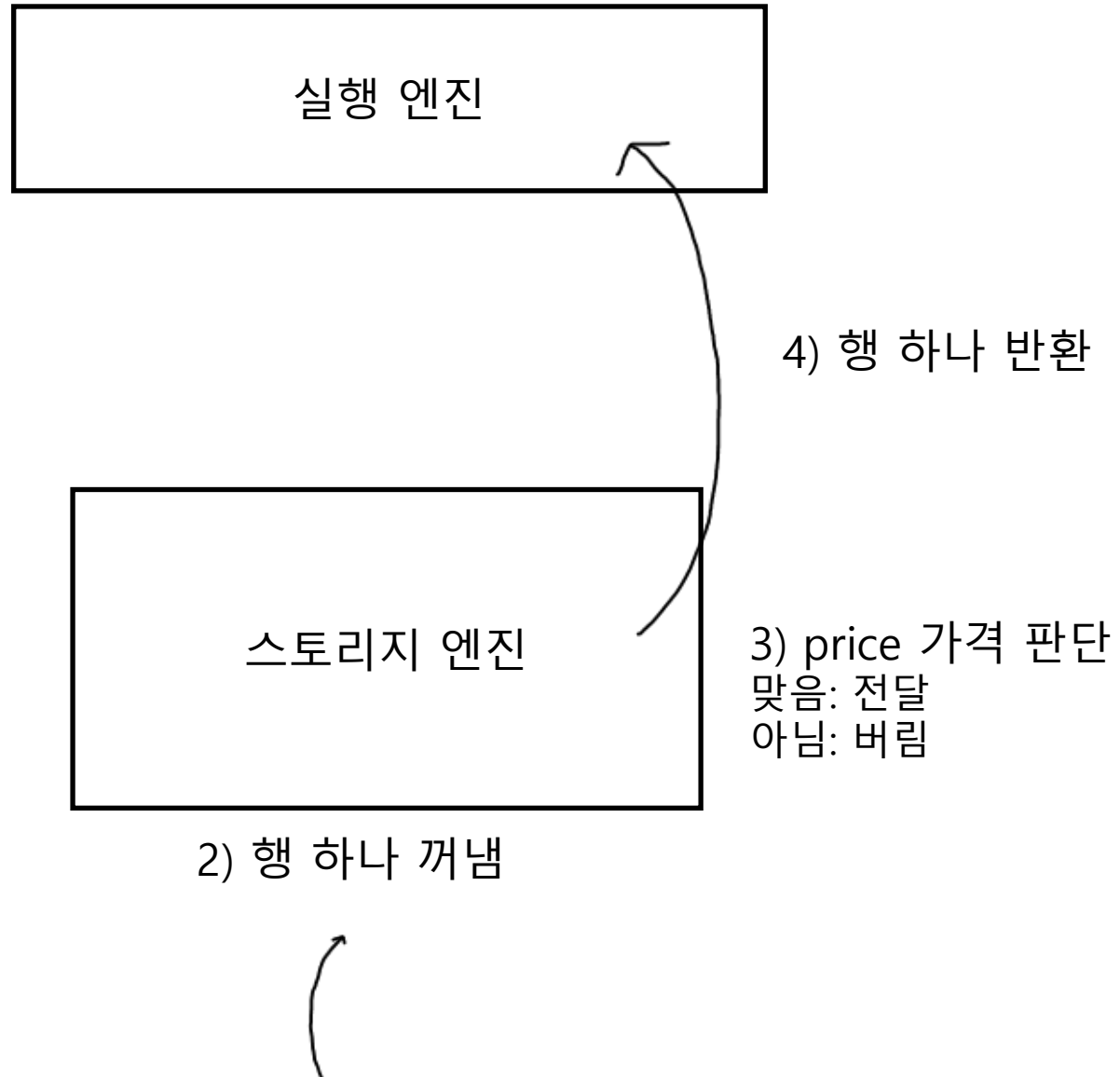
category_id	product_status	created_at	price	pk
11	ACTIVE	2026-08-27	24,000	1
11	ACTIVE	2026-08-28	12,000	2
11	ACTIVE	2026-08-29	45,000	4
11	ACTIVE	2026-08-30	24,000	6
11	ACTIVE	2026-08-31	9,000	5
11	UNACTIVE	2026-08-27	15,000	3
11	UNACTIVE	2026-08-20	8,000	10

스토리지 엔진에서 필터링을 수행한다면?

ICP (Index Condition Pushdown)

실행엔진의 필터 작업을
스토리지 엔진에서 대신 한다!

MySQL의 탐색 구조



3. (category_id, product_status, created_at, price)

category_id	product_status	created_at	price	pk
11	ACTIVE	2026-08-27	24,000	1
11	ACTIVE	2026-08-28	12,000	2
11	ACTIVE	2026-08-29	45,000	4
11	ACTIVE	2026-08-30	24,000	6
11	ACTIVE	2026-08-31	9,000	5
11	UNACTIVE	2026-08-27	15,000	3
11	UNACTIVE	2026-08-20	8,000	10

요구사항의 price 범위가 $10,000 \leq x \leq 13,000$ 이라면?

인덱스

확인해보자!

인덱스

인덱스X

id	select_ty...	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	product	NULL	ref	idx_product_category_id	idx_product_category_id	8	const	413612	1.11	Using where; Using filesort

```
-> Limit: 20 row(s) (cost=62818 rows=20)
    (actual time=682..682 rows=20 loops=1)
    -> Sort: product.created_at, limit input to 20 row(s) per chunk
        (cost=62818 rows=413612) (actual time=682..682 rows=20 loops=1)
        -> Filter: ((product.price between 10000 and 50000) and (product.product_status = 'ACTIVE'))
            (cost=62818 rows=413612) (actual time=0.5..671 rows=91871 loops=1)
            -> Index lookup on product using idx_product_category_id (category_id=11)
                (cost=62818 rows=413612) (actual time=0.446..657 rows=200829 loops=1)
```

인덱스

1. (category_id, product_status, created_at)

id	select_ty...	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	product	NULL	ref	idx_product_category_id,idx_category_status...	idx_category_status_creat...	90	const,const	344330	11.11	Using where

```
-> Limit: 20 row(s)
(cost=78697 rows=20) (actual time=1.18..10.8 rows=20 loops=1)
-> Filter: (product.price between 10000 and 50000)
(cost=78697 rows=38255) (actual time=1.18..10.8 rows=20 loops=1)
-> Index lookup on product using idx_category_status_created_at (category_id=11, product_status='ACTIVE')
(cost=78697 rows=344330) (actual time=1.16..10.7 rows=40 loops=1)
```

2. (category_id, product_status, price, created_at)

id	select_ty...	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	product	NULL	range	idx_category_status_price_created_at	idx_category_status_price...	94	NULL	185848	100.00	Using index condition; Using MRR; Using filesort

```
-> Limit: 20 row(s)
(cost=197400 rows=20) (actual time=2173..2173 rows=20 loops=1)
-> Sort: product.created_at, limit input to 20 row(s) per chunk
(cost=197400 rows=185848) (actual time=2173..2173 rows=20 loops=1)
-> Index range scan on product using idx_category_status_price_created_at over (category_id = 11 AND product_status = 'ACTIVE' AND 10000 <= price <= 50000),
with index condition: ((product.category_id = 11) and (product.price between 10000 and 50000) and (product.product_status = 'ACTIVE'))
(cost=197400 rows=185848) (actual time=23.7..2159 rows=91871 loops=1)
```


인덱스

2. (category_id, product_status, price, created_at)

```
-> Limit: 20 row(s)
(cost=197400 rows=20) (actual time=2173..2173 rows=20 loops=1)
-> Sort: product.created_at, limit input to 20 row(s) per chunk
(cost=197400 rows=185848) (actual time=2173..2173 rows=20 loops=1)
-> Index range scan on product using idx_category_status_price_created_at over (category_id = 11 AND product_status = 'ACTIVE' AND 10000 <= price <= 50000),
with index condition: ((product.category_id = 11) and (product.price between 10000 and 50000) and (product.product_status = 'ACTIVE'))
(cost=197400 rows=185848) (actual time=23.7..2159 rows=91871 loops=1)
```

단일 인덱스(category_id)

```
-> Limit: 20 row(s) (cost=62818 rows=20)
(actual time=682..682 rows=20 loops=1)
-> Sort: product.created_at, limit input to 20 row(s) per chunk
(cost=62818 rows=413612) (actual time=682..682 rows=20 loops=1)
-> Filter: ((product.price between 10000 and 50000) and (product.product_status = 'ACTIVE'))
(cost=62818 rows=413612) (actual time=0.5..671 rows=91871 loops=1)
-> Index lookup on product using idx_product_category_id (category_id=11)
(cost=62818 rows=413612) (actual time=0.446..657 rows=200829 loops=1)
```

3. (category_id, product_status, created_at, price)

id	select_ty...	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	product	NULL	ref	idx_product_category_id,idx_category_status_...	idx_category_status_creat...	90	const,const	353934	11.11	Using index condition

```
-> Limit: 20 row(s)
      (cost=78804 rows=20) (actual time=1.41..9.27 rows=20 loops=1)
-> Index lookup on product using idx_category_status_created_at_price (category_id=11, product_status='ACTIVE'),
    with index condition: (product.price between 10000 and 50000)
      (cost=78804 rows=353934) (actual time=1.4..9.26 rows=20 loops=1)
```

인덱스

1. (category_id, product_status, created_at)

```
-> Limit: 20 row(s)
(cost=78697 rows=20) (actual time=1.18..10.8 rows=20 loops=1)
  -> Filter: (product.price between 10000 and 50000)
    (cost=78697 rows=38255) (actual time=1.18..10.8 rows=20 loops=1)
      -> Index lookup on product using idx_category_status_created_at (category_id=11, product_status='ACTIVE')
        (cost=78697 rows=344330) (actual time=1.16..10.7 rows=40 loops=1)
```

3. (category_id, product_status, created_at, price)

```
-> Limit: 20 row(s)
(cost=78804 rows=20) (actual time=1.41..9.27 rows=20 loops=1)
  -> Index lookup on product using idx_category_status_created_at_price (category_id=11, product_status='ACTIVE'),
    with index condition: (product.price between 10000 and 50000)
    (cost=78804 rows=353934) (actual time=1.4..9.26 rows=20 loops=1)
```

인덱스

프로덕트의 특성에 따라 다르다!

인덱스를 막 걸어도
결국 옵티마이저가 최선의 선택을 하는거 아닌가?

Product

id
price
product_status

```
SELECT *  
FROM product  
WHERE product_status = 'ACTIVE';
```

ACTIVE 85%, UNACTIVE 15%

인덱스

```
EXPLAIN ANALYZE SELECT *  
FROM product  
WHERE product_status = 'ACTIVE';|
```

```
-> Index lookup on product using idx_status_price (product_status='ACTIVE')  
    (cost=302188 rows=2.43e+6) (actual time=1.09..87532 rows=4.25e+6 loops=1)
```

인덱스

```
EXPLAIN ANALYZE SELECT /*+ NO_INDEX(product idx_status_price) */ *  
FROM product  
WHERE product_status = 'ACTIVE';
```

```
-> Filter: (product.product_status = 'ACTIVE')  
      (cost=506257 rows=486612) (actual time=0.455..1580 rows=4.25e+6 loops=1)  
-> Table scan on product  
      (cost=506257 rows=4.87e+6) (actual time=0.426..1289 rows=5e+6 loops=1)
```

약 68배 차이!!!!!!!!!!!!

인덱스

근데 왜 인덱스를 선택할까??

인덱스

```
-> Index lookup on product using idx_status_price (product_status='ACTIVE')  
    (cost=302188 rows=2.43e+6) (actual time=1.09..87532 rows=4.25e+6 loops=1)
```

```
-> Filter: (product.product_status = 'ACTIVE')  
    (cost=506257 rows=486612) (actual time=0.455..1580 rows=4.25e+6 loops=1)  
    -> Table scan on product  
        (cost=506257 rows=4.87e+6) (actual time=0.426..1289 rows=5e+6 loops=1)
```

인덱스

다음 시간에

